

From Frozen Pose to Live Dance: Stochastic Browser Fingerprinting for Robust Risk-Based Authentication

Anonymous Author(s)

Abstract

Risk-Based Authentication (RBA) leverages browser fingerprinting to compare device traits from current and past sessions, helping websites detect risky logins and reduce unnecessary multi-factor authentication prompts. However, current browser fingerprinting is predominantly static and effectively operates as a one-shot snapshot, rendering it fragile to malicious replay and emulation such as phishing attacks. Attackers can replay browser fingerprints obtained from phishing to impersonate users across websites, undermining RBA. In this paper, we study the characteristics of simple content rendering and generation in Canvas, WebGL and Audio. To fundamentally improve the robustness of RBA, based on our findings, we are the first to introduce Stochastic Browser Fingerprinting (S-BFP), a novel browser fingerprinting methodology that embeds controlled randomness or "nonce" to defend against such attacks on RBA. Our key insight is that S-BFP produces controllable different browser fingerprints, even when executed on the same machine rendering identical characters and shapes, creating a unique, robust and replay-resistant device identity. We empirically demonstrate that S-BFP can effectively defend against replay and scripted emulation attacks across major browsers and hardware on more than 100 devices, maintaining stable identification for benign users with negligible overhead.

1 Introduction

Risk-Based Authentication (RBA) has become a common approach for minimizing unnecessary authentication hurdles while strengthening account security [20, 39, 41, 42]. For instance, during login, servers assess risk based on implicit signals, such as device and browser properties, and respond when anomalies are detected. Among these signals, browser fingerprinting (e.g., Canvas, WebGL, Audio, etc.) is particularly compelling as it provides granular, device-specific evidence without necessitating explicit user interaction.

However, state-of-the-art browser fingerprinting used in RBA is vulnerable [26]. Attackers can harvest, package, and

replay victims' browser fingerprints to mimic and impersonate trusted devices across different sites, which can bypass RBA checks. For instance, Lin et al. [26] proposed a new phishing attack to record and replay the browser fingerprints to bypass commercial RBA systems. One of the root causes of such replay attacks is that RBA relying on one-shot, static snapshot browser fingerprints exposes a vulnerability: *such one-shot, static browser fingerprints can be easily reproduced, and thus susceptible to replay and scripted emulation.*

Yet, existing defenses against browser fingerprinting primarily aim to prevent unique user identification by enforcing randomization or uniformity across browser fingerprints [30, 32, 46]. While such approaches can fundamentally mitigate the replay attacks mentioned above, these defenses undermine the core assumption underlying the use of browser fingerprinting in RBA, i.e., the ability to reliably distinguish users. As a result, browser fingerprinting becomes unsuitable for RBA, leading to a substantial degradation in its performance [15, 42, 51]. To fundamentally defend against such attacks, browser fingerprinting must move beyond static snapshots toward a deployment-feasible and adversary-robust paradigm that preserves stability for legitimate users while substantially increasing the cost of reproduction for attackers.

In this paper, we propose the first practical and fundamental defense against replay attack [26] on RBA relying on browser fingerprinting. We introduce Stochastic Browser Fingerprinting (S-BFP), a new approach that generates controllable and different browser fingerprints even when executed on the same physical device, while preserving stable observable characteristics for reliable user identification. S-BFP primarily focuses on browser fingerprinting features such as Canvas text strings, Web Graphics Library (WebGL) graphic rendering, and Audio, which are widely used in state-of-the-art browser fingerprinting and exhibit high entropy [9, 46]. Note that these three can also combine with other browser fingerprinting features to identify users. In the following three paragraphs, we outline the fundamental concepts and methodology for implementing S-BFP to mitigate the aforementioned replay attacks. For the sake of clarity, we shall collectively refer to tasks in-

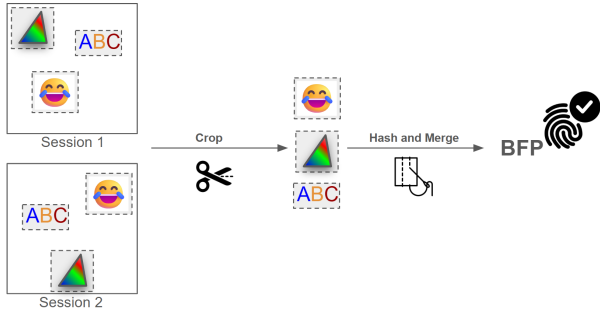


Figure 1: Illustration example of Canvas-based browser fingerprinting in S-BFP for defending against replay attacks in RBA. In each RBA session, the server generates and propagates a random rendering position to the client. The client renders the canvas accordingly and propagate the entire canvas output back to the server. Using the nonce-derived positional information, the server deterministically extracts the fingerprinting primitive (e.g., by cropping the rendered region) and computes its hash for authentication.

volving Canvas text, WebGL graphics, and audio processing as “rendering tasks.”

In traditional BFP systems, once a rendering task is selected by the server, it remains static for every subsequent user login. In contrast, with S-BFP, while the geometric shapes and text content within the rendering task remain constant for each new login attempt, their positions on the Canvas are randomized. Effectively, the content undergoes a random translation (shift) on the canvas for each session. During each login, the position parameters are generated in real-time by the server. Upon completion of the rendering, the server mandates that the client device transmit the data of the entire Canvas back to the backend. On the backend, the server uses the current position parameters to crop the pixel data corresponding to the content’s bounding box from the full canvas. This extracted data serves as the final BFP result. If this result matches the user’s initial registration data, authentication is successful; otherwise, it fails.

Fundamentally, the implementation of S-BFP prevents adversaries from bypassing verification by stealing a single instance of user rendering data. Since the position coordinates required by the server change with every login attempt, an attacker can only submit a stolen, historical canvas. When this outdated canvas is transmitted to the backend, the server will not find the correct graphics at the newly expected coordinates, resulting in authentication failure. Furthermore, attackers cannot simply replay the final cropped BFP result, as the system enforces the transmission of the full canvas data. For details on the system architecture, please refer to §4.

We use Canvas fingerprinting as a concrete example to illustrate our design as shown in the Figure 1. A server-generated nonce, dynamically selecting a random rendering position

within the canvas, is for the browser fingerprinting, and this position is refreshed for each RBA session. The client renders the canvas accordingly and propagates the entire canvas output to the server. Upon receipt, the server uses the nonce-derived positional information to deterministically locate and extract the fingerprints such as by using cropping, compute its hash, and compare the hash result with the reference browser fingerprints associated with the user’s identity. This reference browser fingerprint is established during an initial trusted interaction—such as the user’s manual authentication—when the server first collects and stores the baseline browser fingerprints. A similar approach is employed for WebGL- and Audio-based browser fingerprinting in S-BFP, detailed in §3. Through this process, S-BFP preserves fingerprint stability for legitimate users while preventing replay attacks based on static fingerprints.

A key design assumption of S-BFP is that *objects rendered at different randomized positions can yield identical hash values or rendering results for RBA*. However, prior research [46] has shown that floating-point operations in rendering can introduce subtle discrepancies in rendered outputs. Such discrepancies could potentially violate this assumption, thereby undermining the reliability of matching browser fingerprints and threatening the practical usability of S-BFP in RBA.

We conduct the first systematic study to examine whether, and under what conditions, rendering discrepancies are introduced in S-BFP when the nonce is applied. Specifically, we investigate the impact of various “translation-like” transformations across different modalities: we examine the spatial translation of individual characters and text strings within the Canvas, the translation of rendered triangles across the WebGL coordinate system, and the temporal scheduling of audio snippets (which serves as a functional analogue to spatial translation for audio signals). We consider two types of translations: pixel-level translations, in which objects are shifted by integer pixel offsets, and subpixel-level translations, in which objects are shifted by fractional offsets within a single pixel.

Our key findings are as follows. (1) *For Canvas and Audio, pixel-level translations do not introduce any changes in the resulting hash values. However, for WebGL, while integer translations remain stable in most scenarios, discrepancies can still arise due to its internal floating-point precision and hardware-specific rasterization logic.* (2) *Subpixel-level translations may introduce hash differences; however, these differences exhibit structured and deterministic patterns that can be pre-characterized by the server. Specifically, we observe that the rendering output manifests as a finite set of discrete patterns rather than arbitrary values. It is important to clarify that we do not propose a simplified analytical model to predict the exact hash for a given coordinate. Since predicting the specific hash value at an arbitrary position would effectively require re-executing the entire rendering pipeline. As a result, even if an attacker is aware of these patterns, exploiting them*

to mount replay attacks remains significantly more difficult.

For example, suppose that subpixel translations induce a finite set of rendering patterns $\{a_1, a_2, a_3, a_4\}$. The server can control the fractional components of the translation coordinates to ensure that the client consistently renders a designated pattern (e.g., by locking the output to a_1), thereby maintaining a stable identifier across authentication sessions. More sophisticated strategies may enforce a prescribed sequence across sessions (e.g., $a_1 \rightarrow a_2 \rightarrow a_3 \dots$). Crucially, such control information is not observable by the attacker. As a result, reproducing valid fingerprints requires not only replaying prior outputs but also correctly anticipating the server-specified rendering state, which substantially increases the difficulty of both replay and even corresponding adaptive attacks. The details will be further introduced in §3.

Building upon these observations, we design and implement S-BFP, a new browser fingerprinting system that provides robust protection for RBA against replay attacks. One notable observation is that *Canvas- and Audio-based browser fingerprinting exhibit high stability for unique user identification, whereas WebGL-based fingerprints are occasionally less stable under translation*. This instability arises because WebGL relies on vertex position interpolation to compute fragment colors. During translation, floating-point precision effects can slightly alter the interpolation results, leading to variability in the rendered output. Despite this behavior, WebGL remains a strong candidate for inclusion in S-BFP due to its rich rendering pipeline and high discriminative potential. We leave a deeper investigation of WebGL stability and mitigation strategies to future work.

We evaluate S-BFP through a real-world deployment and user study. The testing system was deployed on a cloud server, and participants from a university Computer Science department were invited to take part in the evaluation. In total, we evaluated the reliability of S-BFP on more than 100 devices, including a subset of mobile devices. All the tests are with standard ethical guidelines discussed in the Ethical Considerations section. Our results demonstrate that S-BFP can effectively defend against existing replay attacks on RBA [26], i.e., 0% attack success rate, while preserving accurate user identification. In particular, S-BFP achieves 100% identification accuracy across all evaluated sessions, thereby maintaining the practical utility of RBA. Moreover, S-BFP introduces only negligible computational and communication overhead compared to conventional browser fingerprinting approaches.

We further analyze adaptive attack strategies, such as stochastic fingerprint brute-force attacks, and demonstrate that such attacks are fundamentally infeasible for bypassing our defense. We also discuss the limitations of S-BFP and outline directions for future work.

Stochastic Browser Fingerprinting (S-BFP). We introduce S-BFP, the first practical browser fingerprinting framework that embeds server-controlled stochasticity into fingerprint generation. S-BFP produces controllably different fingerprints

on the same device across sessions while preserving stable, device-specific identity for benign users, thereby rendering replayed fingerprints ineffective.

To sum up, this paper makes the following contributions:

- *Stochastic Browser Fingerprinting (S-BFP):* We introduce S-BFP, the first practical browser fingerprinting framework that embeds server-controlled nonce into fingerprint generation. S-BFP produces controllably different fingerprints on the same device across sessions while preserving stable, device-specific identity for benign users, thereby rendering replayed fingerprinting attack ineffective for RBA.
- *New Observations of Existing Browser Fingerprinting Features:* We are the first to systematically study how nonce-driven spatial translation affect rendering stability and show that integer-pixel translations preserve identical hash values, while subpixel translations introduce structured, deterministic variations. These variations can be fully controlled and recovered by the server but remain hidden from attackers, substantially increasing the difficulty of replay and adaptive attacks.
- *Novel System Design:* We design a novel system to transform single-shot and static browser fingerprinting into stochastic one, by randomizing element positions, character ordering. S-BFP operates entirely on primitives already collected by today’s RBA, requiring no new client-side components and incurring no perceptible latency.
- *Experimental Validation:* We conduct a comprehensive evaluation of S-BFP against existing practical replay and scripted emulation. S-BFP significantly reduces attack effectiveness while maintaining strong same-device stability, and introduces only negligible computational overhead.

Code and Data Release. Our code and data are fully released to benefit the research in this domain following the standard of Open Science: <https://anonymous.4open.science/r/S-BFP-316B/>

2 Background and Related Work

2.1 Risk-Based Authentication

As digital threats continue to evolve, authentication mechanisms designed to protect sensitive data and user identities are generally used. The journey from traditional password-based authentication to more sophisticated systems such as Two-Factor Authentication (2FA), Multi-Factor Authentication (MFA), and Risk-Based Authentication (RBA) reflects the growing complexity of adversarial threats and demand for usable, scalable security solutions.

2FA [2, 7, 11–14, 21, 33, 35, 37] emerged as an early improvement over password-only schemes by requiring users to present two distinct forms of evidence (factors) to verify their identity. Although 2FA significantly reduces the risk of credential-based attacks, it still relies on static, pre-defined conditions and does not adapt to dynamic threat contexts.

To address the limitations of 2FA, MFA [31, 40] expanded the model to include additional factors such as biometrics (e.g., fingerprints, facial recognition) or behavioral characteristics (e.g., typing patterns), offering a more layered and adaptive approach to user authentication. MFA has become a cornerstone of enterprise security strategies, particularly with cloud-based applications.

However, both 2FA and MFA primarily operate on deterministic rules, triggering challenges uniformly regardless of user behavior or contextual risk. In response to the shortcomings of static authentication mechanisms, RBA [20, 28, 34, 41] has gained traction as a more intelligent and user-centric solution. RBA continuously assesses contextual and behavioral signals such as browser fingerprinting (§2.2) to dynamically evaluate the likelihood of fraud. This enables systems to make real-time decisions about whether to prompt additional verification steps, allow access, or block a request altogether. However, such powerful RBA is vulnerable to replay attack, where the attacker can record the browser fingerprinting and use that to impersonate the users to bypass RBA through phishing attack [26]. In this paper, we propose a novel stochastic browser fingerprinting for robust RBA which can fundamentally defend against the attacks.

2.2 Browser fingerprinting

Browser fingerprinting (BFP) [9, 24, 47] is a technique used to identify users based on a combination of attributes, collected from their browsers, such as cookie [16–18, 23, 25, 38], Canvas [9, 29], etc. Such attributes can be accessed through browser Application programming interfaces (APIs) [44], which serve as interfaces exposing various objects and functions to web applications. The majority of APIs are generally freely accessible to any JavaScript code running within the browser context [9, 24]. Then, the collected attributes can exhibit high entropy and are often unique per user, allowing them to function as quasi-identifiers for purposes such as risk-based authentication (RBA) [3, 4, 43], as well as widely deployed mechanisms in online tracking and anti-bot detection [27, 47].

However, BFP is a double-edged sword [45]: the properties that make it effective for convenience (e.g., application in RBA) also introduce critical attack surfaces. Specifically, the attackers can leverage these uniquely identifying features to replicate a victim’s BFP, thereby evading RBA systems. For instance, Lin et al. [26] proposed an automated pipeline capable of extracting and constructing website-specific fingerprinting, allowing attackers to precisely emulate a target user’s

BFP when interacting with specific web services. Nevertheless, these prior attacks heavily rely on a strong assumption: *the BFP used in RBA is static or invariant for a given user, device, and browser instance—such that replaying a previously captured fingerprint is sufficient to bypass the RBA system*. In this work, we challenge this foundational assumption by providing a novel dynamic BFP that varies even within the same browser and device of a single user. More details can be found in §3.

2.3 Anti Browser Fingerprinting

There are two streams of approaches in defending against browser fingerprinting in general, which are randomization and uniformity. The former, adopted by PriVaricator [30] and some browser add-ons [1], adds noise to the fingerprinting results so that an adversary cannot obtain an accurate fingerprint each time. However, according to prior work [8, 32], such randomization-based defense can be defeated if the adversary fingerprints the browser multiple times and averages the results. Thus, Tor Browser explicitly prefers the latter, i.e., uniformity, over randomization in its design document [32]. However, all these defenses mitigate browser fingerprinting by degrading or disabling functionality, which in turn often renders RBA ineffective and is therefore impractical. These practical shortcomings motivate the need to advance fingerprinting techniques: specifically, to exploit richer rendering signals (e.g., Canvas and WebGL) in ways that preserve legitimate functionality while substantially improving the security and robustness of RBA. Thus, in this paper, we propose a novel stochastic browser fingerprinting, which can defend against the attack [26] on RBA and keep the same RBA benign performance.

2.4 Threat Model for Prior Attacks

In this part, we introduce the threat model of prior attacks, which leverage the browser fingerprinting to attack RBA [26]. The adversary is a remote attacker who can host attacker-controlled website and induce victims to visit them (e.g., via phishing). The adversary can execute JavaScript in the victim’s browser, harvest credentials and site-equivalent static browser fingerprinting features, and reproduce those fingerprint values in a controlled environment (e.g., via API hooking, extension, or proxy) [9, 26]. Additionally, the adversary is capable of injecting these collected browser fingerprinting into requests made to the target service to facilitate authentication bypass. In this paper, we propose the first defense challenging the fundamental assumptions of prior attack, i.e., static browser fingerprinting. With our newly proposed S-BFP, such attack can be defended.

3 Methodology

Existing RBA leverages browser fingerprints captured as static snapshots or identity of client. However, such fingerprints can be obtained by phishing [26] and subsequently replayed to bypass RBA. To systematically mitigate this vulnerability, it is necessary to move beyond “static” fingerprinting and instead generate controllable, different browser fingerprints even when executed on the same physical machine. These fingerprints must preserve identical observable characteristics, while yielding a unique, robust, yet non-replayable device identity. We denote this approach as *stochastic browser fingerprinting (S-BFP)*.

In this paper, we use three representative fingerprinting features—Canvas, WebGL, and Audio—which are widely used in state-of-the-art browser fingerprinting and exhibit high entropy [9, 46]. Specifically, we systematically explore two domains—visual rendering and audio: 2D and 3D rendering (i.e., Canvas and WebGL) and audio-based fingerprinting. Note that these three can also combine with other browser fingerprinting features to identify users. A key design assumption of S-BFP is that *objects rendered at different randomized positions can yield identical hash values or rendering results for RBA*. In this section, we conduct the first systematic study to examine whether, and under what conditions, rendering discrepancies are introduced in S-BFP when the nonce is applied.

3.1 Canvas

The *HTML5 <canvas> element* [19] provides a programmable bitmap surface for script-driven graphics in the browser. When using the “2d” rendering context, drawing operations are executed at the pixel level and rely on the browser’s underlying graphics stack, including font rasterization, text shaping, anti-aliasing, and platform-specific libraries. Since these low-level components vary across browsers, devices, and operating systems, identical drawing commands can yield subtly different pixel outputs. This variability makes the 2D canvas a high-entropy signal and a widely used primitive for browser fingerprinting [9, 29].

Typical fingerprinting approaches draw carefully chosen content such as English string, extract the resulting pixel data (e.g., via `toDataURL()` or `getImageData()`), and hash it into a compact identifier that is stable per device yet discriminative across clients.

To adapt Canvas fingerprinting to the design of S-BFP, we incorporate a nonce—such as a randomized rendering position—to parameterize the placement of rendered objects and thereby prevent replay attacks. This approach relies on a key assumption: *objects rendered at different randomized positions can yield identical hash values or equivalent rendering results for RBA*. Next, we conduct the first systematic study to examine whether, and under what conditions, rendering

discrepancies are introduced in S-BFP when the nonce is applied. Specifically, we translate rendered objects across the canvas and evaluate the resulting changes in hash values.

3.1.1 Assumption Deep Dive

We consider two types of translations: (1) pixel-level translations, in which objects are shifted by integer pixel offsets, and (2) subpixel-level translations, in which objects are shifted by fractional offsets within a single pixel.

Using the letter ‘A’ as a representative example, we observe no fingerprinting differences when the glyph is translated at pixel-level increments across the entire canvas. In contrast, subpixel-level translations introduce pronounced variations. When translating the glyph horizontally at subpixel granularity, we observe four distinct hash values, each corresponding to a specific interval of the fractional subpixel component. As shown in Figure 2 (a), the subpixel range can be partitioned into four regions, within each of which the resulting hash remains identical.

To further investigate this behavior, we generate all English strings containing the letter ‘A’ and collect the subpixel positions at which each instance of ‘A’ is rendered. We then compute the corresponding hash values across different fractional subpixel components, as shown in Figure 2 (b). The two distributions align precisely, indicating that differences in glyph rendering at different character positions within a string are solely attributable to the subpixel positions produced by the text shaping process.

This new observation regarding structured subpixel behavior further strengthens S-BFP by enabling the server to anticipate and control rendering outcomes, thereby increasing the difficulty for attackers to reproduce valid fingerprints and mount replay attacks.

3.1.2 Security Analysis on Canvas in S-BFP

We assume that the attacker attempts to impersonate a legitimate user by reusing rendering results stolen from a phishing page that executes malicious JavaScript. The attacker may learn the fixed set of strings that the server expects the client to render, but the spatial positions of those strings are randomized on each authentication attempt. Let us give an analysis and estimation of the sample size in the probability space of Canvas strings. Let the Canvas dimensions be $\text{canvas_width} = w$ and $\text{canvas_height} = h$. Let the string set $S = \{s_1, s_2, \dots, s_n\}$ denote the n strings assigned to a particular user, and let l_i be the character length of each s_i ($1 \leq i \leq n$). We firstly consider positions are constrained to integer pixel coordinates and that once an anchor position for a string is fixed, the entire string placement is determined.

Counting distinct configurations induced by positional randomness yields a large configuration space. If the server requires n distinct vertical (row) positions, the number of or-

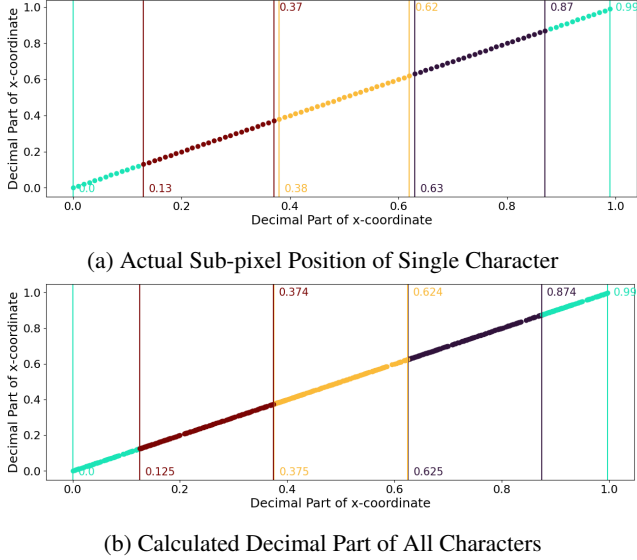


Figure 2: Subpixel-level translation on Canvas. The x-axis represents the fractional (decimal) component of the subpixel translation, and the y-axis denotes the corresponding hash value. (a) The letter ‘A’ is rendered on the canvas and translated horizontally using a subpixel step size of 0.001. (b) English strings containing the letter ‘A’ are rendered on the canvas, and the subpixel fractional positions of each occurrence of ‘A’ are collected. Notably, the resulting variation distributions are perfectly aligned.

dered vertical choices is $P_y = h \cdot (h-1) \cdot (h-2) \cdots (h-n+1)$, which for $h \gg n$ is well approximated by $P_y \approx h^n$. For horizontal (column) placements, each string s_i has approximately $w - l_i$ feasible integer x -positions, so the total number of horizontal combinations is $P_x = \prod_{i=1}^n (w - l_i) \approx w^n$ when $w \gg l_i$. The total number of integer-pixel configurations is therefore

$$\begin{aligned}
 P &= P_x \cdot P_y \\
 &= \left(\prod_{i=1}^n (w - l_i) \right) \cdot (h \cdot (h-1) \cdots (h-n+1)) \quad (1) \\
 &\approx w^n h^n = (wh)^n.
 \end{aligned}$$

Let the adversary have m previous renderings in a configuration space of size P ; then their success probability is

$$\Pr[\text{success}] \leq \frac{m}{P} \approx \frac{m}{w^n h^n}.$$

Because P grows exponentially in n , modest increases in the number of strings n or in canvas resolution dramatically reduce the attack success probability.

The integer-pixel model may be further strengthened by introducing controlled **sub-pixel** perturbations, which can cause glyph variants. We have known that each character may be rendered in k visually distinguishable micro-variants all over the positions on Canvas.

For a given string s_i that can assume K distinct visual forms, we treat these K forms as K distinct strings, each explicitly included in the rendering task. For each form, we specify admissible starting positions and inter-character spacing parameters such that, under these positional constraints, each rendered instance consistently preserves its intended visual appearance.

Consequently, the backend verifier only needs to determine whether the expected forms have been received. If all required forms are observed, the corresponding dynamic fingerprint instance can be reliably recognized as the intended browser fingerprint (BFP).

Example. Consider a string “ab”. Let the rendering positions of the two characters be x_a and x_b along the horizontal axis. The integer part $\lfloor x_a \rfloor$ does not affect the glyph shape of character a , while the fractional part $\{x_a\}$ determines its visual variant. Denote $\{x_a\}$ as the fractional component of x_a . Suppose the unit interval $[0, 1)$ is uniformly partitioned into four equal-length subintervals, each corresponding to one shape of a , denoted $\{a_1, a_2, a_3, a_4\}$. Character b follows the same rule with forms $\{b_1, b_2, b_3, b_4\}$. Hence, the composite string “ab” can exhibit $K = K_a \times K_b = 16$ distinct visual configurations.

In general, for a string $s = c_1 c_2 \dots c_m$, where each character c_j has K_j possible shapes determined by the fractional position $\{x_j\}$ of its horizontal coordinate, the total number of visual configurations is

$$K_s = \prod_{j=1}^m K_j.$$

Each form $s^{(k)}$ maintains its structural integrity within its own stability interval of position parameters, $\{x_j\} \in I_j^{(k)}$, $j = 1, \dots, m$. For example, the shape $a_2 b_2$ corresponds to $\{x_a\}, \{x_b\} \in (0.125, 0.375)$, while the integer offsets of x_a and x_b remain unconstrained, as they are already handled by the integer-pixel translation mechanism discussed earlier.

The backend verification logic only needs to confirm that the expected forms have been rendered and received:

$$\text{Verify}(s) = \begin{cases} \text{true}, & \text{if all required forms are observed;} \\ \text{false}, & \text{otherwise.} \end{cases}$$

Additionally, the system does not need to render all possible forms for each string. Instead, both the frontend and backend can agree on a predefined subset of forms to generate and verify. By selectively rendering only a few representative forms with high visual stability or information gain, the system significantly reduces computational and rendering overhead while preserving the entropy and robustness of the fingerprint. This selective strategy thus achieves a balance between performance and discriminative power.

Sub-pixel strategies effectively extend the placement domain from integer lattice points to a denser sampling of the continuous canvas; in practice, combining position randomization with per-character micro-variants renders exact bitwise replay far more costly for the attacker.

These counting arguments show that even when the adversary knows the string set S , per-session positional randomization (and optional micro-variant amplification) expands the adversary’s search space exponentially. Replay attacks based on a finite corpus of captured renderings therefore have a negligible success probability unless the attacker collects an astronomically large number of samples or leverages additional correlated signals (e.g., server-side cookies, IP/geolocation, or privileged server state). In practice, the server can tune $(w, h, n, \{l_i\}, k)$ to reach a desired security margin while balancing usability and robustness to natural rendering variability.

3.2 WebGL

WebGL (Web Graphics Library) is a browser-based JavaScript API for GPU-accelerated 2D and 3D rendering. WebGL has been shown to provide high entropy and thus, is a popular feature for browser fingerprinting [9]. Owing to device- and implementation-level differences, identical WebGL programs can produce subtly different visual outputs across devices [46]. This is the main property leveraged for traditional WebGL-based browser fingerprinting.

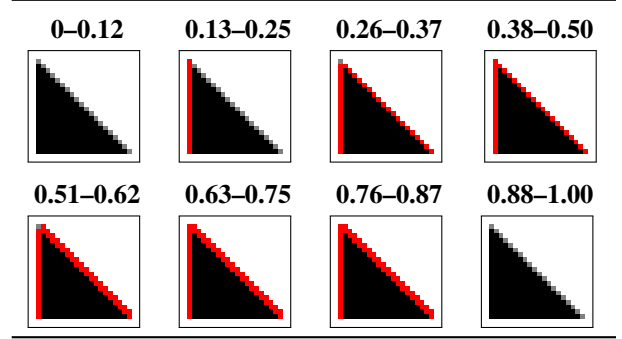
Points, lines, and triangles form the fundamental primitives of WebGL rendering. To investigate the behavior of WebGL graphics under translation, we focus our study on the basic triangle. Our findings reveal that when a WebGL-rendered triangle is translated across the Canvas, it remains stable under pixel-level offsets in most scenarios. However, instabilities still arise under certain conditions, specifically when dealing with varying colors or specific geometric configurations. For sub-pixel level translations, the rendering results exhibit significantly higher instability.

These instabilities primarily stem from precision loss in floating-point calculations—a phenomenon extensively discussed in existing literature [46]. We provide a detailed theoretical explanation of these underlying causes in the subsequent sections. Due to these inherent instabilities, WebGL cannot currently be directly integrated into the S-BFP in its current form. Nonetheless, the potential for addressing these challenges remains a valuable direction for future work.

3.2.1 Illustration of the Observation

Position. We utilize WebGL to render triangles with fixed geometry. The rendered triangle is then translated along the x - and y -axes by subpixel offsets. After each translation, we crop the rendered region containing the triangle and compute pixel-wise differences relative to the initial rendering. Detailed pixel

Table 1: Schematic of an isosceles right triangle rendered in black undergoing successive subpixel translations. Each panel depicts the triangle shifted by the indicated fractional-pixel offset. Pixels highlighted in red represent differences relative to the initial rendering; within each interval, the rendered pixels remain identical. Notably, translations in the range 0.88–1.00 reproduce the initial configuration.



differences under subpixel translations are shown in Table 1.

As observed, different subpixel translation ranges lead to distinct pixel-level renderings, while translations in the range 0.88–1.00 reproduce identical pixel values for the same triangle rendered with a fixed color. We identify two primary factors that contribute to these discrepancies. Given these root causes and the corresponding subpixel translation information, the server can deterministically recover the WebGL-based fingerprinting. This satisfies the fingerprinting requirements of S-BFP.

Factor 1: Pixel Position Rule. During the *rasterization stage*, WebGL maps continuous geometric primitives onto the framebuffer’s discrete pixel grid. Representative algorithms—such as the *Bresenham line algorithm* [22] and the *Digital Differential Analyzer (DDA)* [5]—approximate continuous lines by computing discrete pixel positions so as to minimize deviation from the ideal geometry. Under different subpixel translations, the optimal discrete approximation of a line may cover different sets of pixels, which constitutes one of the root causes of the pixel-level variations observed in Table 1. However, this approximation optimization process is deterministic: given the precise translation parameters, the resulting pixel coverage can be fully recovered by the server. This satisfy the requirement of S-BFP.

Factor 2: Anti-Aliasing Algorithms. Modern GPUs employ anti-aliasing to smooth polygon edges by blending multiple samples within each pixel. At triangle–background boundaries, this process produces transitional pixels with intermediate (e.g., gray) color values. Under subpixel translations, the set of samples intersecting the primitive changes, leading to subtle variations in edge color and opacity. Such anti-aliasing can also be simulated during standard browser fingerprinting.

Color. Gradient fills are commonly used to color triangles. To illustrate color-level discrepancies, we render a triangle with a three-primary-color gradient. During rasterization, the color of each pixel inside the triangle is computed as a weighted blend of the three vertex colors, where the weights—given by barycentric coordinates—depend on the pixel’s position relative to the triangle’s vertices.

For a triangle with vertices v_1, v_2, v_3 and corresponding colors C_1, C_2, C_3 , the interpolated color C_f at a fragment f is computed as:

$$C_f = \lambda_1 C_1 + \lambda_2 C_2 + \lambda_3 C_3,$$

$$\lambda_1 + \lambda_2 + \lambda_3 = 1, \quad \lambda_i \geq 0, \quad i = 1, 2, 3.$$

Here, C_1, C_2, C_3 represent the colors assigned to the triangle’s vertices. The coefficients λ_1, λ_2 , and λ_3 are the barycentric coordinates of fragment f with respect to the triangle, determined by the fragment’s relative position within the triangle:

$$\lambda_1 = \frac{\text{area}(v_f v_2 v_3)}{\text{area}(v_1 v_2 v_3)}, \quad \lambda_2 = \frac{\text{area}(v_f v_3 v_1)}{\text{area}(v_1 v_2 v_3)}, \quad \lambda_3 = \frac{\text{area}(v_f v_1 v_2)}{\text{area}(v_1 v_2 v_3)}$$

where v_* indicate the vertex. This mechanism enables WebGL to generate smooth per-fragment color gradients directly in hardware, without requiring explicit per-pixel computations in the fragment shader.

Because the barycentric coefficients (λ) are computed using floating-point arithmetic, their fractional values vary slightly under subpixel translations. These small numerical differences, when quantized to discrete RGB values, can result in ± 1 changes in pixel. Note that this phenomenon is especially severe on Apple devices. Consequently, the interior RGB values of a *varying-color* triangle are not perfectly consistent but exhibit subtle fluctuations in response to subpixel movement with a cycle pattern similar to Canvas and WebGL position introduced above. We use the CPU to calculate theoretical RGB value to check the color difference with different subpixel translation of WebGL real rendering. The results are shown in Figure 3.

3.2.2 Security Analysis on WebGL in S-BFP

The evaluation of attacks and defenses in the WebGL part is basically consistent with the logic in the Canvas part; it is essentially like replacing individual characters with triangles. We will first analyze the sample space of these triangles. Once the position of a single vertex A is fixed, the entire triangle is determined. Suppose vertex A can take any pixel-aligned coordinate within the Canvas, and there are P such possible positions in total. Then, each triangle has P possible configurations. For a rendering task containing n triangles, the entire rendering state space consists of P^n possible combinations.

Non-Overlapping Constraint. In practice, triangles must remain non-overlapping; otherwise, backend cropping produces

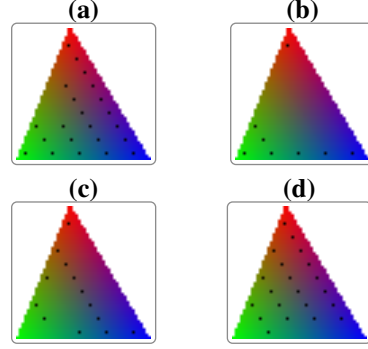


Figure 3: The mismatched pixels color inside the triangle at different subpixel translation. Black pixels indicate where the CPU-computed theoretical RGB values differ from the GPU-obtained results.

mixed pixels, preventing consistent hashing and breaking verification. Thus, only configurations with all triangles disjoint are considered.

Each triangle i cover b_i discrete pixels on a canvas with P total pixel positions. As triangles are placed sequentially, the number of valid configurations can be roughly expressed:

$$N = P \times (P - b_1) \times (P - b_2) \times \cdots \times (P - b_{n-1}),$$

which, for large $P \gg b_i$, simplifies to $N \approx P^n$.

This approximation ignores boundary effects but effectively captures how smaller triangles (smaller b_i) permit more distinct placements, while larger ones restrict n .

In each authentication session the WebGL rendering is randomized, so an adversary must present *exactly one* matching sample from a vast configuration space to pass verification. Our backend extracts local hashes at the expected triangle positions, concatenates the n segment hashes, and computes a composite hash; any mismatch in placement yields a different hash and is flagged as suspicious.

If the attacker has collected m legitimate renderings, the probability of a successful replay is only $\Pr[\text{success}] = \frac{m}{P^n}$ since those m samples represent only m out of the P^n possible configurations. Collecting a large m is infeasible due to requiring victims to produce distinct renderings.

An attacker could, in principle, attempt to brute-force the rendering space, but increasing n quickly makes exhaustive search infeasible. When the system assigns multiple triangle sets and performs r rendering rounds per authentication, the effective configuration space $P_1^{n_1} \times P_2^{n_2} \times \cdots \times P_r^{n_r}$, grows exponentially, making successful impersonation practically impossible. Going a step further, we can adopt the same approach as in the Canvas part, extending the triangle positions from integer pixels to **sub-pixel precision**, achieving results consistent with those in the Canvas section.

3.3 Audio

Browsers generally leverage the *Web Audio API* for in-browser audio synthesis and processing. Differences in browser implementations and underlying platforms introduce consistent variations in rendered waveforms, making audio a useful signal for browser fingerprinting [9]. By rendering and hashing simple offline audio graphs, (i.e., hashing aggregated waveform amplitude values), prior work [9, 10, 36, 50] obtains identifiers that are stable per device yet distinct across environments, complementing graphics-based fingerprints.

To achieve the properties of S-BFP, we introduce randomness into the waveforms using a server-generated nonce that is unknown and unpredictable to the attacker. The resulting randomized waveforms are propagated to the client, where they are used to generate audio-based fingerprints that are subsequently collected by the server. Possession of the nonce allows the server—but not the attacker—to reconstruct the original waveform, enabling accurate and consistent fingerprinting.

Specifically, we construct each waveform by concatenating multiple distinct waveform segments (e.g., segments with different frequencies), randomly permuting these segments, selectively inserting silent intervals (i.e., 0 waveform amplitude) between certain segments, and then reconnecting the resulting sequence to form the final waveform used by S-BFP. On the server side, the nonce deterministically specifies the segmentation, permutation, and silencing operations, allowing the server to precisely recover the original waveform.

Fundamental hardness of reconstruction for the attacker, even with known permutation. Audio-based browser fingerprinting in S-BFP is fundamentally difficult for an attacker to reconstruct due to the combined effects of different distinct waveform segments and selectively inserting silent intervals. Even if the attacker knows the different distinct waveform segments, accurately reconstructing the original signal remains challenging because the reconstruction process is numerically order-sensitive, analogous to floating-point arithmetic.

Specifically, in floating-point summation, the order of operations affects the final result; for example, $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$ where $\oplus$ denotes floating-point addition [49]. Similarly, when different distinct waveform segments are connected, small numerical and boundary-level discrepancies accumulate in an order-sensitive manner, leading to divergent reconstructed signals. This inherent sensitivity introduces fundamental obstacles for the attacker, making precise waveform reconstruction infeasible even with knowledge of the transformation.

3.3.1 Security Analysis on Audio in S-BFP

In the audio module, each user is required to generate n audio snippets. These snippets are separated by $n - 1$ gaps, which we denote by $\mathbf{s} = (s_1, s_2, \dots, s_{n-1})$. We can adjust these gaps and

analyze the sample space, similar to the previous section. We will not elaborate further here.

4 System Design

Based on the primitives we studied in Section 3. We build the S-BFP RBA system with replay resistant capability.

4.1 Overall Design

Figure 4 depicts the design of S-BFP. Essentially, S-BFP generates a *Device_ID* following two steps:

(1) *Static Fingerprint*. Obtain a traditional static browser fingerprint from the client device as the preliminary device identification. (2) *Stochastic Fingerprint*. Based on the preliminary device identification, generate a set of personalized *primitives* further perturbed spatially. Challenge the client device to render the primitives and calculate a *stochastic browser fingerprint* as the *Device_ID*.

Note that S-BFP provides two-fold defense against malicious replays suffered by traditional browser fingerprint: (1) The set of primitives differs per device (i.e., different devices render different strings and audio waveforms of different frequency), and is dynamically designated in the authentication process. Thus the fingerprint cannot be easily collected in advance without communication with our server. (2) The spatial perturbation (i.e., strings and triangles rendered at different position in the canvas) makes the challenges different per each authentication session. Previous rendering results cannot be reused.

4.2 Components of S-BFP

S-BFP has 4 major components to realize the aforementioned device identification process - the *Static-BFP module*, *Randomness-Injection module*, *Stochastic-BFP module* and *Device_ID Calculation module*.

(1) **Static-BFP module**. The Static-BFP module is mainly used to preliminarily identify a device to dispatch challenges for further verification. We utilize the well studied traditional browser fingerprint methods with high discriminative power to implement this module (e.g., User-Agent String, WebGL Renderer information, complex 2D images and 3D scenes rendered with Canvas and WebGL). This module is a front-end JavaScript library. The fingerprint result will be collected and hashed on the client device and sent to the verifier.

(2) **Randomness-Injection module**. The Randomness-Injection module generates a deterministic random set of *primitives* (i.e., strings with content or audio wave forms of certain frequency) per device. The primitives are “deterministic” in the sense that the same device always gets the same set of *primitives*, but the primitives are random across devices. To achieve this goal, we use the hashed static fingerprint as the “seed” to feed a deterministic random bit generator (DRBG).

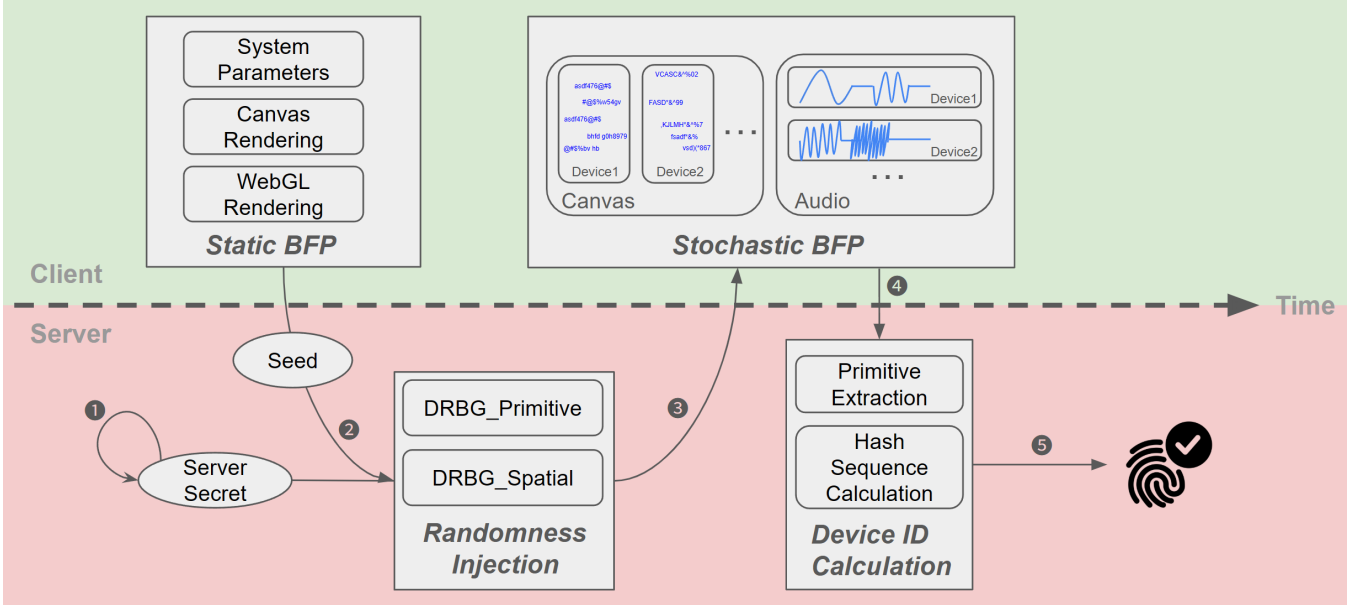


Figure 4: System Overview of S-BFP. S-BFP extends traditional static browser fingerprinting by adding randomized challenges to ensure the fingerprint is fresh and authentic. ❶ The server maintains a secret and updates periodically. ❷ S-BFP obtain a traditional BFP, use it as a seed to combine with the server secret. It then uses the combined input to initialize two deterministic random bits generators. ❸ S-BFP generates random rendering tasks and requests the client to generate content with Canvas and Audio APIs. ❹ The generated content is sent back to the server for analysis and comparison. ❺ S-BFP tries to crops out the primitives and calculates a device ID.

To further prevent an attacker from emulating the generation process with the seed, the verifier also keeps a secret byte sequence (i.e., **server_secret**) of high entropy on the server. We use the concatenated byte sequence of **seed** and **server_secret** to initialize the DRBG for primitive designation (*DRBG_Primitive*),

$$DRBG_Primitive = DRBG(\text{seed} \parallel \text{server_secret})$$

Note that *DRBG_Primitive* always generates an identical random bit sequence given the same **seed** with **server_secret** unchanged. **server_secret** can be updated to invalidate any registered stochastic browser fingerprint to mitigate a security incident like a phishing campaign.

S-BFP utilizes another DRBG (*DRBG_Spatial*) to generate spatial perturbations. *DRBG_Spatial* is initialized as,

$$DRBG_Spatial = DRBG(\text{timestamp} \parallel \text{server_secret})$$

where **timestamp** is the timestamp on the verifier’s server when an authentication request is received. In this manner, *DRBG_Spatial* encodes the current timestamp as a freshness guarantee.

We implement *DRBG_Primitive* and *DRBG_Spatial* in compliance with the *NIST SP 800-90A HMAC-DRBG (SHA-256)* specification [6].

(3) Stochastic-BFP module. This module is also a front-end JavaScript library that fetches the parameters of the

rendering challenge from the verifier’s server, providing the *seed*. We challenge the client device to render content with 3 sets of API - *Canvas* and *Audio*:

- **Canvas.**

- The primitives are strings of random content with a trailing emoji.
- Spatial perturbation are random (x, y) coordinates in the Canvas.

- **Audio.**

- The primitives are sinusoid waveform snippets of different frequency.
- Spatial perturbation are inter-snippet silent gaps of random length.

The whole canvas or *AudioBuffer* is dumped and sent back to the verifier’s server for verification together with the *seed*.

(4) Device_ID Calculation module. The server records the expected spatial location of the primitives before dispatching the challenges to the client device (i.e., bounding boxes for the strings, triangles and boundaries of the audio snippets). After receiving the rendering result, the *Device_ID Calculation* module retrieves the records and try to extract the primitives at the desired location. A hash sequence is computed for each of the primitive and combined to calculate a final hash as the *Device_ID*.

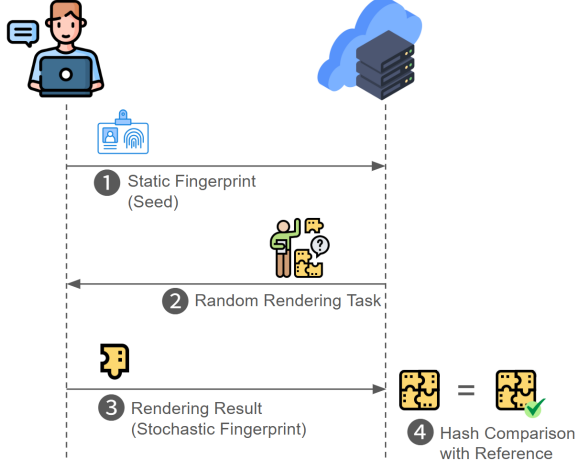


Figure 5: Authentication Process

4.3 Authentication Process

In this section, we will summarize how each component of S-BFP function in a complete authentication cycle. Figure 5 shows the keys steps in an authentication cycle of S-BFP.

(1) Obtaining the static fingerprint. The client will run the *Static-BFP module* of S-BFP which is embedded in the website front end. The *Static-BFP module* will obtain a traditional BFP on the client device and send the hash value to the authentication server. We call this hash value the "*Seed*". It serves as the initial self-claimed device identity.

(2) Generation of a random rendering task. Upon receiving the *seed*, the *Randomness-Injection module* will combine it with a secret bytes maintained on the server(*server secret*) and use it to deterministically generate a set of random primitives and random position parameters (using *DRBG_Primitive* and *DRBG_Spatial*). The client will be requested to generate content specified by the primitives and position parameters.

(3) Fulfillment of the random rendering task. The client will run the *Stochastic-BFP module* locally with the primitives and position parameters received from the server. Under the hood, the *Stochastic-BFP module* will rendered the requested contents using the Canvas and Audio APIs and send the original rendering results back to the server together with the *seed*.

(4) Device ID computation and comparison. The rendering result or *Stochastic fingerprint* will be processed by the *Device_ID Calculation module*. Specifically, it crops out each primitive at the expected position specified by the position parameters sent to the client in step 2. A hash value will be computed for each of the primitives. The sorted list of primitive hashes will be hashed again and used as the final *Device ID*. S-BFP compares the calculated device ID with historical device IDs associated with the same *seed*. If it matches with historical records, the authentication succeeds if the user also provides the correct credentials later. Otherwise, we will

further require 2FA.

5 Evaluation

In this section, we evaluate S-BFP in terms of its defense effectiveness and its performance comparison to original browser fingerprinting such as overhead, effectiveness of unique user identification.

5.1 Effectiveness

We first introduce the evaluation setup. The testing platform was deployed on a cloud server, and participants from a university Computer Science department were invited to take part in the evaluation. Note that during data collection, all participants were informed of the experimental purpose, and participation was entirely voluntary. The collected data were anonymized, securely stored on the server, and used exclusively for scientific research.

In total, S-BFP was evaluated on 213 devices including multiple representative operating systems (OS) and browsers. Note that such OS and browsers are also used in the prior browser fingerprinting research [9, 46]. The details are shown in Table 3.

Table 2 demonstrates that S-BFP achieves exceptional reliability, particularly with the 100% stability rate in Audio and 99.0% in Canvas. The minor instability in Canvas was limited to only two specific devices (one Windows PC and one Android smartphone). We attribute these discrepancies to non-integer device pixel ratios or browser zoom levels rather than any fundamental flaw in the S-BFP methodology. Furthermore, a small fraction of samples failed to yield valid fingerprinting data due to external factors such as network latency, script-blocking extensions, or hardware-level API permission restrictions, which prevented the successful execution of the rendering tasks.

Table 2: Stability Analysis of Different Modalities.

Feature	Stable / Total	Unstable	Rate (%)
Audio	206 / 206	0	100.0
Canvas	196 / 198	2	99.0
WebGL	177 / 193	16	91.7

5.2 Comparison with Original Browser Fingerprinting

We compare S-BFP with conventional browser fingerprinting along two primary dimensions: the ability to uniquely identify users and system overhead.

The ability to uniquely identify users. We evaluate this component along two dimensions: entropy and hash collision

Table 3: Distribution of User Environments Across Different Combinations

Combination	Count	Percentage
Windows + Chrome	42	19.7%
Windows + Messenger ¹	40	18.8%
Android + Messenger ¹	36	16.9%
Windows + Edge	28	13.1%
iOS + Messenger ¹	16	7.5%
Android + Chrome	10	4.7%
Android + Messenger ²	8	3.8%
macOS + Chrome	8	3.8%
macOS + Safari	7	3.3%
iOS + Safari	6	2.8%
Unknown + Messenger ¹	4	1.9%
Windows + Firefox	3	1.4%
Linux + Chrome	2	0.9%
Android + Firefox	1	0.5%
macOS + Edge	1	0.5%
Linux + Firefox	1	0.5%

¹ Built-in browser of major IM applications (e.g., WeChat).

² Branded mobile browsers from IM providers (e.g., QQ Browser).

rate. The dynamic element of S-BFP is realized exclusively through translation operations. Specifically, whereas original browser fingerprinting renders an object at a fixed position, S-BFP preserves the rendered content and varies only its spatial position. As a result, the intrinsic entropy of the fingerprint remains unchanged.

System overhead. Crucially, S-BFP introduces only a single additional rendering pass with dynamic coordinate parameters to the standard fingerprinting process. As a result, the system achieves enhanced security against replay attacks without incurring significant computational overhead compared to traditional, static BFP methods.

As shown, S-BFP incurs approximately twice the overhead of traditional browser fingerprinting. This increase is expected, as S-BFP executes the fingerprinting procedure twice, as described in §4. Despite this increase, the absolute execution time remains modest and well within practical limits. Our evaluation further shows that the performance of S-BFP comfortably meets the requirements of common display refresh rates (e.g., 60Hz), indicating that the added overhead does not impact user experience.

6 Adaptive Attacks

To illustrate the security guarantees of S-BFP, we enumerate several representative attack strategies in this section. The

difficulty of successfully launching these attacks increases with the sophistication of the attack, and each is mitigated by specific properties of the stochastic fingerprint mechanism and the careful design of S-BFP. The reader can refer to §4.3 for a normal authentication process without any malicious party.

Attack 1: Naive Replay. The simplest form of attack is

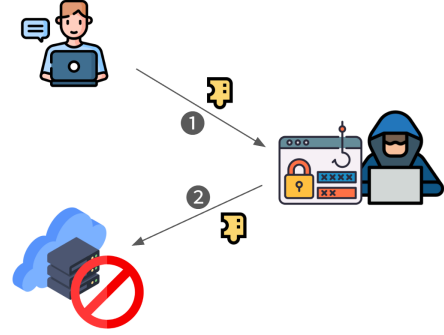


Figure 6: Naive Replay

that the attacker tries to duplicate the fingerprinting code on the legitimate website and deploy to a phishing website, in the hope of collecting the fingerprint directly from the victim. This is also the replay attack suffered by traditional static fingerprinting method as discussed in previous work [26]. This is simply impossible for S-BFP. First, The fingerprinting code is a parameterized function that needs designation of primitives and position parameters from the server. The primitives are personalized for each client and the position parameters are time sensitive (i.e., the rendering task differs per device and per authentication session). The attacker does not know what content to render without knowledge of the state of the *Randomness Injection module* on the server.

Attack 2: Stochastic Fingerprint Brute Force. In this

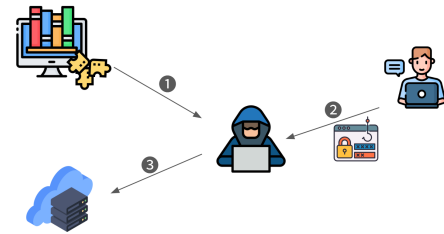


Figure 7: Stochastic Fingerprint Brute Force

attack, the attacker circumvents obtaining the state of the *Randomness Injection module*. Instead, he tries to enumerate the complete parameter space of stochastic fingerprint rendering and build an offline dictionary of all possible rendering results. The attacker will collect the credentials and *seed* from the victim. When he tries to login with the legitimate credential and *seed*, the servers sends the correct rendering task. The at-

tacker then retrieves the pre-computed rendering results from the dictionary and replies to the server. This approach is also impossible. First, the total parameter space is prohibitively large. The reader can refer to §3.1.2 and §3.3.1 for numerical analysis. Second, the correct primitive can only be rendered on the client’s physical device. The rendering results will have subtle differences across devices, which is the fundamental property that supports browser fingerprinting.

Attack 3: Static Fingerprint Brute Force. Another ap-

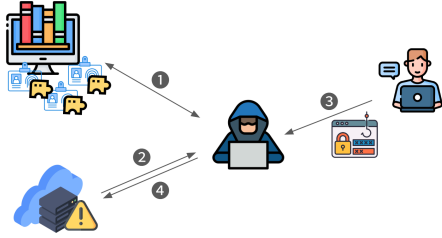


Figure 8: Static Fingerprint Brute Force

proach to circumvent obtaining the state of the *Randomness-Injection module* is to exhaustively build a dictionary for the static fingerprint. Recall that the static fingerprint (i.e., *seed*) is used as an unreliable self-claimed device identity in S-BFP (§4.3). Provided with the *seed*, the server will generate the random rendering task and send the parameters to the client. The attacker will try to build a complete dictionary for seeds, repeatedly probing the authentication server for primitives, and store the *seed* $\leftrightarrow$ *primitive* mapping. When the attacker launches the phishing campaign, he collects the credentials and *seed* from the victim. Then he retrieves the primitives associated with the *seed* from the dictionary and obtains the rendering result from the victim’s device. When he wants to login, he will emulate a fresh rendering result from his records. This is a more sophisticated approach, however, it still won’t work. First, the *seed* is calculated from values either in a very large combinatorial space (e.g., user-agent string, timezone, system configurations), or bound to the physical device (e.g., Canvas and WebGL rendered images), thus is not enumerable. Second, continuous probing the authentication server to build a large dictionary is a very prominent behavior pattern that can be detected.

In summary, the attacks increase in sophistication from naive replay to adaptive enumeration, but in each case, the stochastic fingerprint mechanism and session-dependent randomness ensure that all these attacks are effectively mitigated.

7 Discussion

7.1 Incapable for Enhancing Tracking

S-BFP is structurally incapable of enhancing browser-based tracking, and its design intent is strictly confined to strengthening replay resistance within RBA. Unlike traditional fin-

gerprints that rely on cross-session stability, S-BFP replaces stability with verifier-bound, session-specific stochasticity: every fingerprint instance is derived from server-side secrets and per-session challenge randomness, making it non-transferable across services and non-linkable across sessions, even on the same device. The mechanism never introduces new high-entropy identifiers, never emits globally consistent signals, and never increases the observability of low-level attributes beyond what current RBA systems already require. As a result, S-BFP cannot be repurposed to extend tracking horizons or support cross-site identity linkage; its only effect is to raise the cost of replay and scripted emulation attacks within an authenticated session. In short, S-BFP enhances RBA robustness, not tracking power, and is intentionally engineered so that tracking gains are both impossible and explicitly out of scope.

7.2 Limitations and Future Works

First, S-BFP does not aim to defend against the strongest adversaries that fully control or perfectly clone a victim’s environment: if an attacker obtains persistent system-level access or faithfully replicates the victim’s hardware, OS, and browser stack, they can natively execute our probing code and reproduce the same stochastic distribution, which lies outside our remote-attacker threat model. Second, while our evaluation covers major desktop and mobile browsers on commodity GPUs, its platform and deployment scope remain limited; extending S-BFP to a broader range of devices and validating it in large-scale, production RBA deployments is an important direction for future work. Third, S-BFP currently assumes that standard Canvas/WebGL/Audio APIs are available as in today’s RBA setups; systematically studying its interaction with browser-side anti-fingerprinting and privacy mechanisms, and potentially designing first-party-only or RBA-scoped variants, is left to future work. Fourth, S-BFP shows some hash collision on the WebGL. Future work could be further improve the WebGL part in S-BFP.

8 Conclusion

In this paper, we are the first to challenge the foundational assumption of prior attacks on RBA: the reliance on static or single-shot browser fingerprinting, by introducing a novel Stochastic Browser Fingerprinting. Leveraging the new properties of existing browser fingerprinting features uncovered by this research, we design a novel, S-BFP, transforming single-shot and static browser fingerprinting into stochastic one, by randomizing element positions, character ordering, and initialization seeds across Canvas, WebGL, and Audio primitives. Our experiments shows that S-BFP can significantly reduce the attack effectiveness while maintaining strong same-device stability. We hope that our findings can inspire future research into this critical domain.

Ethical Considerations

Human participants. Although our study involves human participants running browser related experiments on their own devices, Institutional Review Board (IRB) is not required for this research. Similar to prior works in this domain [46, 48], our study only collects non-identifiable fingerprinting information: while browser fingerprinting techniques can be used for tracking or profiling, the fingerprint itself—like a cookie—is merely an identifier. It does not contain personally identifiable information (PII). To ensure ethical transparency for our experiments, we adopt a consent-based data collection process. During the signup phase, all participants are presented with an additional consent page clearly informing them of the nature of the fingerprint data being collected. This consent mechanism is integrated into the user interface and is explicitly acknowledged by each participant before data collection begins. Furthermore, all collected fingerprinting data is associated only with anonymized user IDs, and no personal or device-identifiable information is stored or inferred.

Users. S-BFP significantly enhances the account security of end-users by preventing adversaries from bypassing Risk-Based Authentication (RBA) through replay attacks, thereby safeguarding user accounts from unauthorized access and potential data compromise.

Service Providers. For service providers, S-BFP offers a robust, replay-resistant browser fingerprinting mechanism that effectively mitigates the impact of replay attacks on RBA systems. This is achieved while maintaining stable user identification and incurring negligible additional computational and communication overhead.

adversaries. S-BFP increases the cost for attacker to bypass RBA via replay attacks, and practically speaking, even if attacker knows the full content of our paper, he cannot successfully verify it using the simple replay attacks and adaptive attacks discussed in this paper.

To sum up, our study adheres to established ethical standards in human-subjects research involving non-identifiable data, leverages transparent and informed consent, and respects legal privacy requirements.

Open Science

We strictly comply with the open science policy by making our implementation available following ethical standards: <https://anonymous.4open.science/r/S-BFP-316B/>.

References

- [1] Canvas defender. <https://multiloginapp.com/canvasdefender-browser-extension/>.
- [2] Claudia Ziegler Acemyan, Philip Kortum, Jeffrey Xiong, and Dan S Wallach. 2fa might be secure, but it's not usable: A summative usability assessment of google's two-factor authentication (2fa) methods. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, volume 62, pages 1141–1145. SAGE Publications Sage CA: Los Angeles, CA, 2018.
- [3] Nampoina Andriamilanto, Tristan Allard, and Gaëtan Le Guelvouit. “guess who?” large-scale data-centric study of the adequacy of browser fingerprints for web authentication. In *International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing*, pages 161–172. Springer, 2020.
- [4] Nampoina Andriamilanto, Tristan Allard, and Gaëtan Le Guelvouit. Fpselect: low-cost browser fingerprints for mitigating dictionary attacks against web authentication mechanisms. In *Proceedings of the 36th Annual Computer Security Applications Conference*, pages 627–642, 2020.
- [5] H Banbrook. Digital differential analyzer. Technical report, 1963.
- [6] Elaine B Barker, John Michael Kelsey, et al. *Recommendation for random number generation using deterministic random bit generators (revised)*. US Department of Commerce, Technology Administration, National Institute of ..., 2007.
- [7] Jessica Berrios, Elias Mosher, Sankofa Benzo, Cinthya Grajeda, and Ibrahim Baggili. Factorizing 2fa: Forensic analysis of two-factor authentication applications. *Forensic Science International: Digital Investigation*, 45:301569, 2023.
- [8] Yinzhi Cao, Zhanhao Chen, Song Li, and Shujiang Wu. Deterministic browser. In *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, 2017.
- [9] Yinzhi Cao, Song Li, and Erik Wijmans. (Cross-)Browser Fingerprinting via OS and Hardware Level Features. In *Proceedings 2017 Network and Distributed System Security Symposium*. Internet Society, 2017.
- [10] Shekhar Chalise, Hoang Dai Nguyen, and Phani Vadrevu. Your speaker or my snooper? measuring the effectiveness of web audio browser fingerprints. In *Proceedings of the 22nd ACM Internet Measurement Conference*, pages 349–357, 2022.
- [11] Jessica Colnago, Summer Devlin, Maggie Oates, Chelse Swoopes, Lujo Bauer, Lorrie Cranor, and Nicolas Christin. “it's not actually that horrible” exploring adoption of two-factor authentication at a university. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, pages 1–11, 2018.

- [12] Sanchari Das, Andrew Dingman, and L Jean Camp. Why johnny doesn't use two factor a two-phase usability study of the fido u2f security key. In *International Conference on Financial Cryptography and Data Security*, pages 160–179. Springer, 2018.
- [13] Sanchari Das, Bingxing Wang, Zachary Tingle, and L Jean Camp. Evaluating user perception of multi-factor authentication: A systematic review. *arXiv preprint arXiv:1908.05901*, 2019.
- [14] Alexandra Dmitrienko, Christopher Liebchen, Christian Rossow, and Ahmad-Reza Sadeghi. Security analysis of mobile two-factor authentication schemes. *Intel Technology Journal*, 18(4), 2014.
- [15] Antonin Durey. *Leveraging browser fingerprinting to strengthen web authentication*. PhD thesis, Université de Lille, 2022.
- [16] Steven Englehardt and Arvind Narayanan. Online Tracking: A 1-illion-site Measurement and Analysis. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 1388–1401, 2016.
- [17] Steven Englehardt, Dillon Reisman, Christian Eubank, Peter Zimmerman, Jonathan Mayer, Arvind Narayanan, and Edward W Felten. Cookies That Give You Away: The Surveillance Implications of Web Tracking. In *Proceedings of the 24th International Conference on World Wide Web*, pages 289–299, 2015.
- [18] Evercookie. Evercookie. <https://sa.my/evercookie/>.
- [19] Steve Fulton and Jeff Fulton. *HTML5 canvas: native interactivity and animation for the web*. " O'Reilly Media, Inc.", 2013.
- [20] Anthony Gavazzi, Ryan Williams, Engin Kirda, Long Lu, Andre King, Andy Davis, and Tim Leek. A study of {Multi-Factor} and {Risk-Based} authentication availability. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2043–2060, 2023.
- [21] Roger Grimes. The many ways to hack 2fa. *Network Security*, 2019(9):8–13, 2019.
- [22] P Koopman. Bresenham line-drawing algorithm. *Forth Dimensions*, 8(6):12–16, 1987.
- [23] Balachander Krishnamurthy and Craig E Wills. Generating a privacy footprint on the internet. In *Proceedings of the 6th ACM SIGCOMM conference on Internet measurement*, pages 65–70, 2006.
- [24] Pierre Laperdrix, Nataliia Bielova, Benoit Baudry, and Gildas Avoine. Browser fingerprinting: A survey. *ACM Transactions on the Web (TWEB)*, 14(2):1–33, 2020.
- [25] Ada Lerner, Anna Kornfeld Simpson, Tadayoshi Kohno, and Franziska Roesner. Internet Jones and the Raiders of the Lost Trackers: An Archaeological Study of Web Tracking from 1996 to 2016. In *25th USENIX Security Symposium (USENIX Security 16)*, 2016.
- [26] Xu Lin, Panagiotis Ilia, Saumya Solanki, and Jason Polakis. Phish in Sheep's Clothing: Exploring the Authentication Pitfalls of Browser Fingerprinting. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 1651–1668, 2022.
- [27] Zengrui Liu, Jimmy Dani, Yinzhi Cao, Shujiang Wu, and Nitesh Saxena. The first early evidence of the use of browser fingerprinting for online tracking. In *Proceedings of the ACM on Web Conference 2025*, pages 4980–4995, 2025.
- [28] Mohammed Misbahuddin, BS Bindhumadhava, and B Dheeptha. Design of a risk based authentication system using machine learning techniques. In *2017 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computed, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/CBDCOM/IOP/SCI)*, pages 1–6. IEEE, 2017.
- [29] Keaton Mowery and Hovav Shacham. Pixel Perfect: Fingerprinting Canvas in HTML5. In *Proceedings of W2SP*, volume 2012, 2012.
- [30] Nick Nikiforakis, Wouter Joosen, and Benjamin Livshits. Privaricator: Deceiving fingerprinters with little white lies. In *Proceedings of the 24th International Conference on World Wide Web*, pages 820–830, 2015.
- [31] Aleksandr Ometov, Sergey Bezzateev, Niko Mäkitalo, Sergey Andreev, Tommi Mikkonen, and Yevgeni Koucheryavy. Multi-factor authentication: A survey. *Cryptography*, 2(1):1, 2018.
- [32] Mike Perry, Erinn Clark, Steven Murdoch, and Georg Koppen. The design and implementation of the tor browser. <https://www.torproject.org/projects/torbrowser/design/>.
- [33] Thanasis Petsas, Giorgos Tsirantonakis, Elias Athanopoulos, and Sotiris Ioannidis. Two-factor authentication: is the world ready? quantifying 2fa adoption. In *Proceedings of the eighth european workshop on system security*, pages 1–7, 2015.

- [34] Claudy Picard and Samuel Pierre. Rlauth: A risk-based authentication system using reinforcement learning. *IEEE Access*, 11:61129–61143, 2023.
- [35] Ahmad R Pratama and Firman M Firmansyah. Until you have something to lose! loss aversion and two-factor authentication adoption. *Applied Computing and Informatics*, 21(1/2):53–64, 2025.
- [36] Jordan S Queiroz and Eduardo L Feitosa. A web browser fingerprinting method based on the web audio api. *The Computer Journal*, 62(8):1106–1120, 2019.
- [37] Joshua Reynolds, Nikita Samarin, Joseph Barnes, Taylor Judd, Joshua Mason, Michael Bailey, and Serge Egelman. Empirical measurement of systemic {2FA} usability. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 127–143, 2020.
- [38] Franziska Roesner, Tadayoshi Kohno, and David Wetherall. Detecting and Defending Against Third-Party Tracking on the Web. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 155–168, 2012.
- [39] Mariusz Sepczuk and Zbigniew Kotulski. A new risk-based authentication management model oriented on user’s experience. *Computers & Security*, 73:17–33, 2018.
- [40] Tance Suleski, Mohiuddin Ahmed, Wencheng Yang, and Eugene Wang. A review of multi-factor authentication in the internet of healthcare things. *Digital health*, 9:20552076231177144, 2023.
- [41] Stephan Wiefeling, Markus Dürmuth, and Luigi Lo Iacono. More than just good passwords? a study on usability and security perceptions of risk-based authentication. In *Proceedings of the 36th Annual Computer Security Applications Conference*, pages 203–218, 2020.
- [42] Stephan Wiefeling, Luigi Lo Iacono, and Markus Dürmuth. Is this really you? an empirical study on risk-based authentication applied in the wild. In *IFIP International Conference on ICT Systems Security and Privacy Protection*, pages 134–148. Springer, 2019.
- [43] Stephan Wiefeling, Tanvi Patil, Markus Dürmuth, and Luigi Lo Iacono. Evaluation of risk-based re-authentication methods. In *IFIP International Conference on ICT Systems Security and Privacy Protection*, pages 280–294. Springer, 2020.
- [44] WIKI. API. <https://en.wikipedia.org/wiki/API>.
- [45] Shujiang Wu et al. *Double-edged Sword: An in-depth analysis of browser fingerprints for web tracking and bot defenses*. PhD thesis, Johns Hopkins University, 2023.
- [46] Shujiang Wu, Song Li, Yinzhi Cao, and Ningfei Wang. Rendered Private: Making GLSL Execution Uniform to Prevent WebGL-based Browser Fingerprinting. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1645–1660, 2019.
- [47] Shujiang Wu, Pengfei Sun, Yao Zhao, and Yinzhi Cao. Him of many faces: Characterizing billion-scale adversarial and benign browser fingerprints on commercial websites. In *NDSS*, 2023.
- [48] Shujiang Wu, Jianjia Yu, Min Yang, and Yinzhi Cao. Rendering Contention Channel Made Practical in Web Browsers. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3183–3199, 2022.
- [49] Peichen Xie, Yanjie Gao, Yang Wang, and Jilong Xue. Revealing {Floating-Point} accumulation orders in {Software/Hardware} implementations. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 1425–1440, 2025.
- [50] Desheng Zhang, Jianhui Zhang, Youjun Bu, Bo Chen, Chongxin Sun, and Tianyu Wang. A survey of browser fingerprint research and application. *Wireless Communications and Mobile Computing*, 2022(1):3363335, 2022.
- [51] Ye Zhang, Feng Wang, Jianbin Zeng, Lutong Chen, Xuanbo Huang, Zhonghui Li, and Kaiping Xue. User behavior-based dynamic authentication design for enhanced identity security. In *ICC 2025-IEEE International Conference on Communications*, pages 1–6. IEEE, 2025.